SPMT 5708 Homework 3 (Data Manipulation in R)

The process of manipulating datasets is often overlooked in the grand scheme of sports analytics. While many focus on conducting analyses and interpreting results, properly formatted data are necessary before any other steps can take place. R, with the help of the *dplyr* package, has a variety of functions that make data wrangling easier.

3.1 Data Manipulation of Columns and Rows (Select & Filter)

1. Import the *FBS_2025* Excel file into R Studio. Remember to load the *dplyr* package into this R session using the *library* command. If you receive an error message, be sure the package is still installed (*install.packages*).

2. Recall some of the *dplyr* functions we ran during class:

   Use the ***select*** function to create a new dataset called *WPct_Only* that only contains columns pertaining to the schools' names, overall winning percentages, and conference winning percentages. *This function is useful for removing unused variable columns and selecting only those we want to use in our analyses. We can use column numbers or column (variable) names:*

   ```
   WPct_Only=FBS_2025%>%select(School,WPct,ConfWPct)
   WPct_Only
   ```

   We can also use the ***filter*** command to retain certain rows as specified by some logical criteria. In this scenario, we want to create a new dataset containing only ACC teams:

   ```
   ACC_2025=FBS_2025%>%filter(Conf=="ACC")
   ACC_2025
   ```

   This function is useful for only retaining certain subsets of observations. Make sure to note the various operators that can be used in the logical sequences:

Operator	Logical Meaning
==	equal to
!=	not equal to
|	or
&	and
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to

The pipe operator *%>%* can be used to naturally stack functions in *dplyr*. For example, we could combine the previous two functions to create a new subset of data that focuses solely on schools with winning records:

```
Winners_Only=FBS_2025%>%select(School,WPct,ConfWPct)%>%filter(WPct>=.5)
Winners_Only
```

Note that the functions are applied in order from left to right. So, in this example, the data are first subset to the selected columns and then the filter function is applied.

3.2 Other Data Manipulation Techniques (Arrange, Mutate, & Summarize)

1. Now, let's try the ***arrange*** command. In this example, I want you to create a subset of the *FBS_2025* dataset looking at ACC *and* SEC teams. Then, you need to sort them by conference and conference win percentage. To do so, you'll need to run the following code:

   ```
   ACC_SEC=FBS_2025%>%filter(Conf=="ACC" | Conf=="SEC")%>%arrange(Conf,
   ConfWPct)
   ACC_SEC
   ```

   This arranges the ACC and SEC teams first by conference, then by their conference winning percentage. By default, it will go in *ascending* order. However, if we wanted it to go in *descending* order so that the winningest teams were at the top, we could make a slight addition to our code:

   ```
   ACC_SEC=FBS_2025%>%filter(Conf=="ACC" | Conf=="SEC")%>%arrange(Conf,
   desc(ConfWPct))
   ACC_SEC
   ```

2. The ***mutate*** function can be used to create a new column by modifying some of the data we have in other columns. In this particular scenario, let's create a new column called *MOV* showing ACC and SEC teams' average margins of victory (or defeat) by subtracting their opponents' points per game (*OPPG*) values from their own points per game (*PPG*) values.

   ```
   ACC_SEC_NEW=ACC_SEC%>%mutate(MOV=PPG-OPPG)
   ACC_SEC_NEW
   ```

   Note how we simply used the ACC_SEC data frame we created in our previous example instead of running all the filter and arrange codes again.

   Which team in the ACC had the largest average margin of victory (*MOV*)? The lowest? What about in the SEC?

ANSWER: The team with the lowest MOV in the ACC was Syracuse, and in the SEC was Kentucky. The team with the highest MOV in the ACC was Miami (FL) and in the SEC was Ole Miss

3. The last of the main functions in *dplyr* is the ***summarize*** command. This takes the values for a group of data and calculates summary statistics for them. For instance, let's say we were curious which conference (out of all the FBS conferences), on average, played the toughest schedule. We could use the following codes to find that answer:

   ```
   Conf_SOS=FBS_2025%>%group_by(Conf)%>%summarize(ASOS=mean(SOS, na.rm=TRUE))
   Conf_SOS
   ```

   The *group_by* command aggregates the specific rows that we want to summarize; in this case, it's grouping the observations (schools) by the conference (*Conf*) they're in, and then using the *mean* function to find the group averages of the variables we noted (*SOS*). Remember that we've inserted the *na.rm=TRUE* operator to ignore the NAs when calculating the mean.

   So, which conference played the toughest schedule, on average, in 2025?

   ANSWER: The conference with the toughest schedule on average in 2025 was the Big Ten with an average SOS of 5.48.

4. The *mean* function isn't the only one we could use. Let's say we wanted to further find the best (numerically lowest) AP ranking achieved by a team in a given conference. To do this, we simply supply the *min* command to the *AP_High* column, creating a new column called *BestRk*.

   ```
   Conf_Stats=FBS_2025%>%group_by(Conf)%>%summarize(ASOS=mean(SOS, na.rm=TRUE), BestRk=min(AP_High, na.rm=TRUE))
   Conf_Stats
   ```

   Which conference(s) boasted the best ranking?

   ANSWER: The conferences with the best ranking were Big Ten and SEC.

3.3 Merging Datasets

1. Import the *FBS_2024* dataset into R Studio, retaining that name. Now, let's stack (bind) it beneath the *FBS_2025* dataset we previously imported to form a larger, more comprehensive dataset. To do this, we'll rely on the *bind_rows* function from *dplyr*.

FBS_Full=bind_rows(FBS_2025,FBS_2024)

2. Next, let's say that instead of stacking them, we simply wanted to join the *WPct* column from *FBS_2024* to *FBS_2025* for purposes of comparing teams' winning percentages across the two seasons. For this, we can use one of the *join* functions from *dplyr*. First, though, we need to reduce *FBS_2024* down to the two columns that are needed:

   FBS_2024=FBS_2024%>%select(School, WPct)

   Now, let's complete the merge using the *left_join* function:

   FBS_Multi=left_join(FBS_2025,FBS_2024,by="School")

3. You should notice a couple of things. First, because the column names for *WPct* were duplicated after the merge, they automatically received *.x* and *.y* labels for 2025 and 2024, respectively. Second, there are two schools (Delaware and Missouri State) that were not in the FBS in 2024, so they have values of *NA* for *WinPct.y*.

   To address the former of these, we can use the ***rename*** function in *dplyr* to appropriately label these columns. Then, we could rearrange them using the *select* function and the appropriate column numbers:

   FBS_Multi=FBS_Multi%>%rename("WPct_25"="WPct.x", "WPct_24"="WPct.y")
   FBS_Multi=FBS_Multi%>%select(1:4,16,5:15)

   If we wanted to remove the two rows containing Delaware and Missouri State since they weren't present in both seasons, we could use the ***slice*** function to remove those rows by their corresponding row number (should be 71 and 72):

   FBS_Multi=FBS_Multi%>%slice(-c(71:72))

   Alternatively, we could have used *filter* to only select rows that did not have those two schools' names, or the *inner_join* function on the front end to only retain rows that had matching values for *School* in <u>both</u> the *FBS_2024* and *FBS_2025* datasets.

   Now that the dataset has been cleaned and reorganized, *create a new column* showing the difference between these programs' win percentages in 2024 and 2025. Which *dplyr* function did you use? What was the change in FSU's win percentage from 2024 to 2025? In Clemson's?

   ANSWER: I used the function mutate. From 2024 to 2025, FSU had a difference of 0.250, meaning that they won more in 2025. Clemson had a difference of -0.176, meaning that they won less in 2025 season.

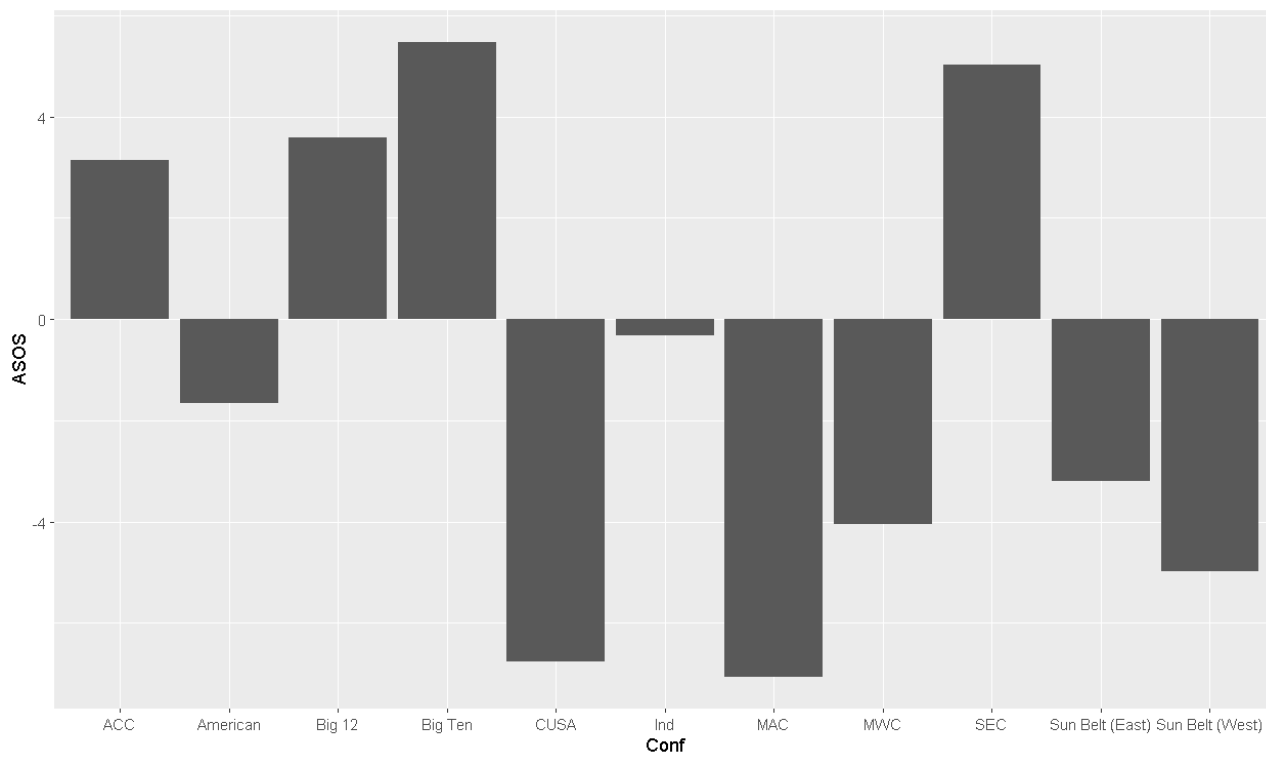
3.4 Brief Overview of Data Visualization

Using the *ggplot2* package (make sure *install.packages* and *library* have been used), you can create a wide array of graphics. This functions in this package use a consistent syntax for building data graphics incrementally (layer by layer).

1. Install and read *ggplot2* into your R platform. Then, use the following code to create a simple ***bar graph*** visualizing the differences in the conferences' average strength of schedule (*ASOS*) measures:

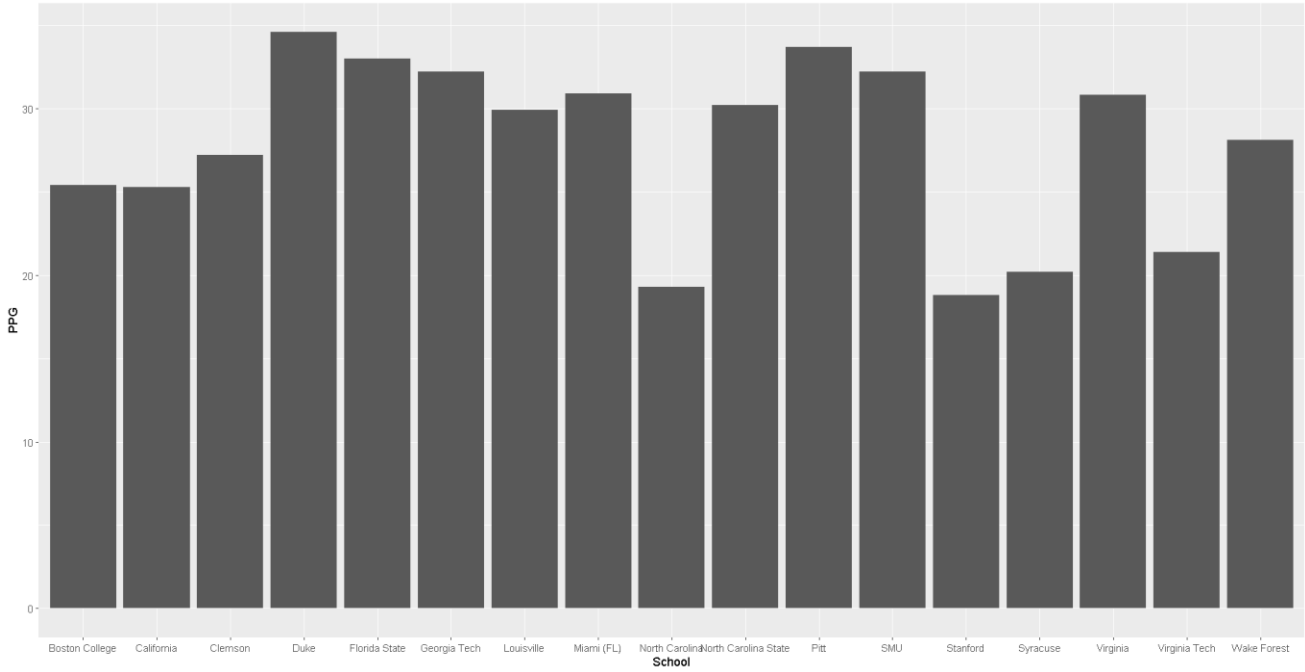
   ggplot(Conf_Stats,aes(x=Conf,y=ASOS))+geom_col()

   Your plot should appear in the bottom right quadrant of R Studio. It can be exported (saved as a new file or copied to a clipboard for pasting) or zoomed in on using the buttons on the interface. **Paste your plot below when finished.**



   Within the *ggplot* structure, the *aes* code stands for "*aes*thetic mapping" while the *geom_col()* command is associated with a "*geom*etric" object (in this case the *col*umns of a bar chart). Additional codes can be added to change the color or size of elements in the visual. *Note that ggplot visuals use + signs instead of pipes (%>%) to add additional layers.*

2. Create a bar graph of your own using the data (or a subset of the data) in *FBS_2025.* Simply modify the previous code to contain the data object and variable column that are of interest to you. **Paste your personal graph below**.



Submit your R Script (code) *and* this completed Word document (with highlighted questions answered) to the drop box on Canvas before the start of the next class.